LAB1 -ACTIVITY

Name:	
CMS:	
Date	

ASSEMBLY LANGUAGE

Language:

It is a source or way of communication. In programming it is a set of instructions which produces a desired output.

Assembler:

It is a native translator of Assembly language.it translates the high level language into middle language (Assembly Language).

Introduction and Definition of Assembly Language:

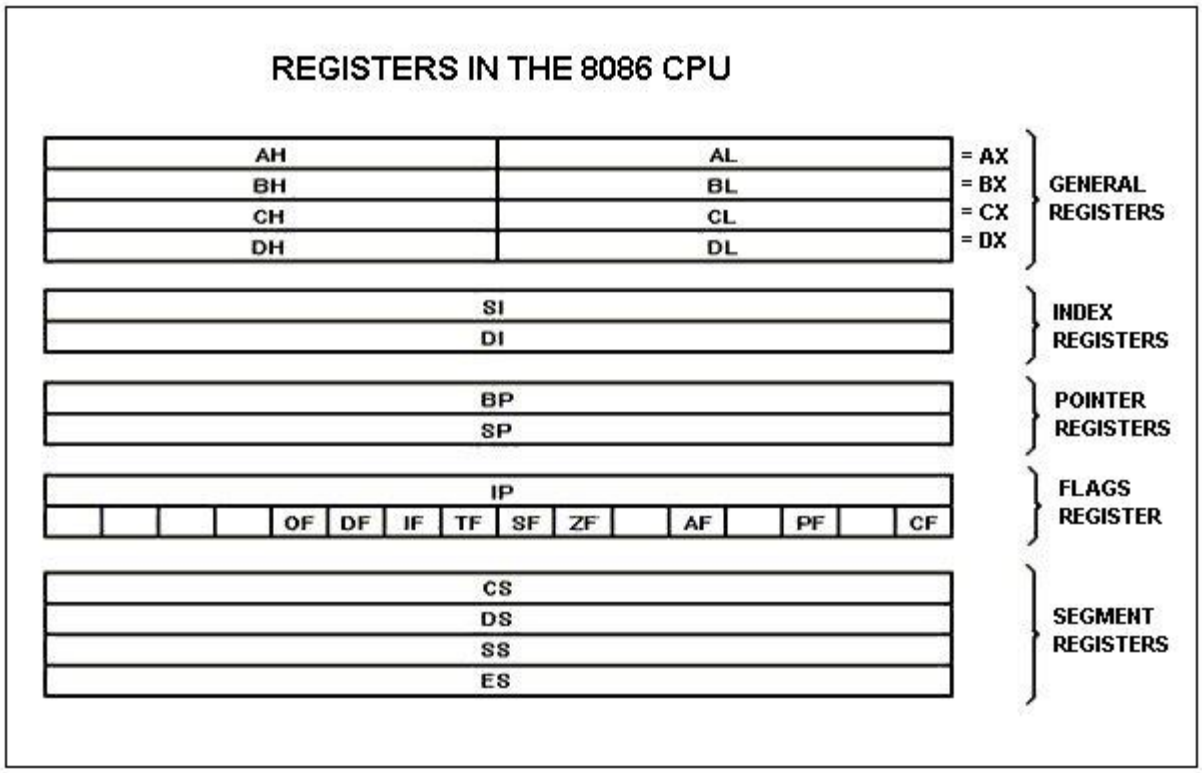
An assembly language is a low-level programming language for microprocessors and other programmable devices. An assembly language implements a symbolic representation of the machine code needed to program a given CPU architecture. Assembly language is also known as assembly code.
Assembly language is very close to the machine language. It is very faster than high level language. It also has the direct access of memory of CPU.

Why learn Assembly Language:

- If you study computer engineering or computer science, you may likely to be asked to write embedded programs. They are shorter programs stored in a small amount of memory for a single purpose devices.
- Hardware manufactures creates drivers for equipment they sell. Device drivers are the drivers that translates general operating system commands into specific references to hardware specific details.
- Computer game consoles(X-Box) require their software to be highly optimized (compressed) for small code size and fast execution it allocates access of internal memory directly. Game programmers are experts at writing code that takes full advantage of hardware futures in a targeted system.
- It can use **mnemonic** than numeric operation **code** and it also provides the information of any error in the **code**. it can use mnemonic than numeric operation **code** and it also provides the information of any error in the **code**.
- This **language** helps in specifying the symbolic operand that means it does not need to specify the machine address of that operand.
- It is not required to keep track of memory locations.
- It allows complex jobs to run in a simpler way.
- It requires less instruction to get the result.

REGISTORS

Inside the 8086 Central Processor Unit (CPU)



GENERAL PURPOSE REGISTERS

8086 CPU has 8 general purpose registers; each register has its own name:

AX - the accumulator register (divided into **AH / AL**):

1. Generates shortest machine code
2. Arithmetic, logic and data transfer
3. One number must be in AL or AX
4. Multiplication & Division
5. Input & Output

BX - the base address register (divided into **BH / BL**).

CX - the count register (divided into **CH / CL**):

- Iterative code segments using the LOOP instruction
- Repetitive operations on strings with the REP command
- Count (in CL) of bits to shift and rotate

DX - the data register (divided into **DH / DL**):

1. DX:AX concatenated into 32-bit register for some MUL and DIV operations
2. Specifying ports in some IN and OUT operations

SI - source index register:

1. Can be used for pointer addressing of data
2. Used as source in some string processing instructions
3. Offset address relative to DS

DI - destination index register:

1. Can be used for pointer addressing of data
2. Used as destination in some string processing instructions
3. Offset address relative to ES

BP - base pointer:

1. Primarily used to access parameters passed via the stack
2. Offset address relative to SS

SP - stack pointer:

1. Always points to top item on the stack
2. Offset address relative to SS
3. Always points to word (byte at even address)
4. An empty stack will had SP = FFFEh

SEGMENT REGISTERS

CS - points at the segment containing the current program.

DS - generally points at segment where variables are defined.

ES - extra segment register, it's up to a coder to define its usage.

SS - points at the segment containing the stack.

Although it is possible to store any data in the segment registers, this is never a good idea. The segment registers have a very special purpose - pointing at accessible blocks of memory.

Segment registers work together with general purpose register to access any memory value. For example if we would like to access memory at the physical address **12345h** (hexadecimal), we could set the **DS = 1230h** and **SI = 0045h**. This way we can access much more memory than with a single register, which is limited to 16 bit values.
The CPU makes a calculation of the physical address by multiplying the segment register by 10h and adding the general purpose register to it (1230h * 10h + 45h = 12345h):

```
 12300
+ 0045
-------
 12345
```

The address formed with 2 registers is called an **effective address**.
By default **BX, SI** and **DI** registers work with **DS** segment register;
BP and **SP** work with **SS** segment register.
Other general purpose registers cannot form an effective address.
Also, although **BX** can form an effective address, **BH** and **BL** cannot.

SPECIAL PURPOSE REGISTERS

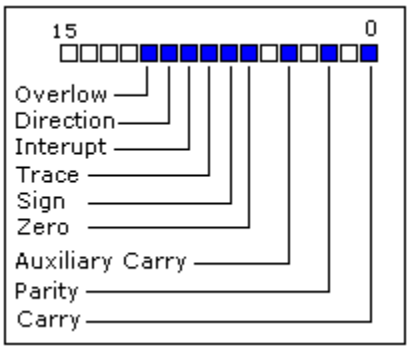
IP - the instruction pointer:

1. Always points to next instruction to be executed
2. Offset address relative to CS

IP register always works together with **CS** segment register and it points to currently executing instruction.

FLAGS REGISTER

Flags Register - determines the current state of the processor. They are modified automatically by CPU after mathematical operations, this allows to determine the type of the result, and to determine conditions to transfer control to other parts of the program.
Generally you cannot access these registers directly.



1. **Carry Flag (CF)** - this flag is set to **1** when there is an **unsigned overflow**. For example when you add bytes **255 + 1** (result is not in range 0...255). When there is no overflow this flag is set to **0**.
2. **Parity Flag (PF)** - this flag is set to **1** when there is even number of one bits in result, and to **0** when there is odd number of one bits.
3. **Auxiliary Flag (AF)** - set to **1** when there is an **unsigned overflow** for low nibble (4 bits).
4. **Zero Flag (ZF)** - set to **1** when result is **zero**. For non-zero result this flag is set to **0**.
5. **Sign Flag (SF)** - set to **1** when result is **negative**. When result is **positive** it is set to **0**. (This flag takes the value of the most significant bit.)
6. **Trap Flag (TF)** - Used for on-chip debugging.
7. **Interrupt enable Flag (IF)** - when this flag is set to **1** CPU reacts to interrupts from external devices.
8. **Direction Flag (DF)** - this flag is used by some instructions to process data chains, when this flag is set to **0** - the processing is done forward, when this flag is set to **1** the processing is done backward.
9. **Overflow Flag (OF)** - set to **1** when there is a **signed overflow**. For example, when you add bytes **100 + 50** (result is not in range -128...127).